

Video 5: Summary and Key Takeaways

Best practices for AI-assisted Python coding

The Repeatable Vibe Coding Playbook: 8 Steps

1	2	3	4	5	6	7	8
Define the Goal	Ask for a Plan	Convert to Tasks	Implement One Task	Add Validation Hooks	Generate Tests	Debug and Review	Refactor and Repeat
One clear paragraph. What does it do? Who uses it? What does it NOT do? No ambiguity.	Ask AI for architecture before writing any code. Get components and suggested build order.	Break each component into individual tasks small enough for one prompt each.	Use Template 3. One task per prompt. Never bundle. Test before moving to the next.	Ask AI to annotate assumptions and uncertainties inline as code comments.	Ask AI to write unit tests immediately after each function is generated.	Run tests. Feed failures back using Template 4. Read code. Apply review checklist.	Every 3-4 tasks: check for duplication, resolve TODO hooks, update PROJECT_SPEC.md.

Apply this loop to any Python project. The playbook works for a tax calculator, an expenses tracker, and anything you build next.

The Cost of Overreliance on AI

Speed without comprehension is not
productivity—it's technical debt in motion

Three Risks Every Vibe Coder Must Manage

Risk 1: Shallow Understanding

If you cannot explain a function you generated, in plain language, to a colleague, you do not own it.

Fix: Ask AI to explain every piece of generated code. Read it until you can paraphrase it. Add your own comments. Slow down to go fast.

Risk 2: Accumulated Technical Debt

Stubs left unresolved. Duplicate logic never cleaned up. Validation hooks never verified. Each one is a future bug at compound interest.

Fix: Treat every TODO and ASSUMPTION comment as a debt entry. Resolve on a schedule—not "when I have time."

Risk 3: False Sense of Productivity

Code that runs is not the same as code that works correctly, securely, and sustainably.

Fix: Measure productivity by tests passing and features verified, not by lines of code generated. Output is not the same as quality.

Rule of Thumb: If you could not maintain this code without the AI, you built it too fast.

The Balance: Speed With Comprehension

The goal of this course is not to help you generate code quickly. It is to help you build things that work, that you understand, and that you can improve.

What to Keep

- Using AI to move faster on well-understood, repeatable tasks
- Using AI as a collaborator: plan together, implement together, review yourself
- Using AI to learn, ask for explanations, not just code
- The iterative loop, the templates, the playbook

What to Watch

- Moving so fast that you stop understanding what you've built.
- Treating AI as a search engine for correct code. It isn't.
- Stacking features before earlier features are verified.
- Forgetting that you, not the AI, are responsible.

AI is a multiplier. It amplifies the habits of the developer using it—good habits and bad ones equally.

What to Do Next: Three Practice Projects

Practice Project 1

Extend Your Tax Calculator

- Add support for different tax years (2023/24 vs. 2024/25).
- Add a command-line interface, accept income as user input.
- Export results to a formatted text file.
- Use the full 8-step playbook: plan first, scaffold, implement, test.

Practice Project 2

Build the Expenses Tracker from Scratch

- Add expense entries (amount, category, date).
- Show a summary by category.
- Save and load data from a CSV file.
- No hand-holding—full playbook, independent build.

Practice Project 3

Choose Your Own Project

- Pick something you actually need or want to build.
- Start with the Planning Prompt. No code before a plan.
- Apply every technique from this course.
- Notice where AI helps and where you need to take control.

The skill of AI-assisted development compounds quickly. Each project will be faster, cleaner, and more controlled than the last.

Your Goal: After three projects, you should be able to explain every function you've built. That's mastery.

Everything This Course Has Given You

Video 1: Mental Model

AI is a probabilistic generator, not a software engineer. You are the Director. Know when to use it and when to take manual control.

Video 2: Building the Skill

Staged prompting: Plan → Requirements → Tasks → Implement. The iterative loop. Four reusable prompt templates.

Video 3: Testing and Debugging

Generate tests immediately. Feed failures back. Apply the 5-point review checklist. Own everything AI writes.

Video 4: Advanced Techniques

PROJECT_SPEC.md for context. Intentional stubs as roadmaps. Deduplication audits. Validation hooks for traceability.

The 8-step playbook ties it all together. Define → Plan → Tasks → Implement → Hooks → Tests → Debug → Refactor. Repeat.

Welcome to the new way of building software. Now go build something.